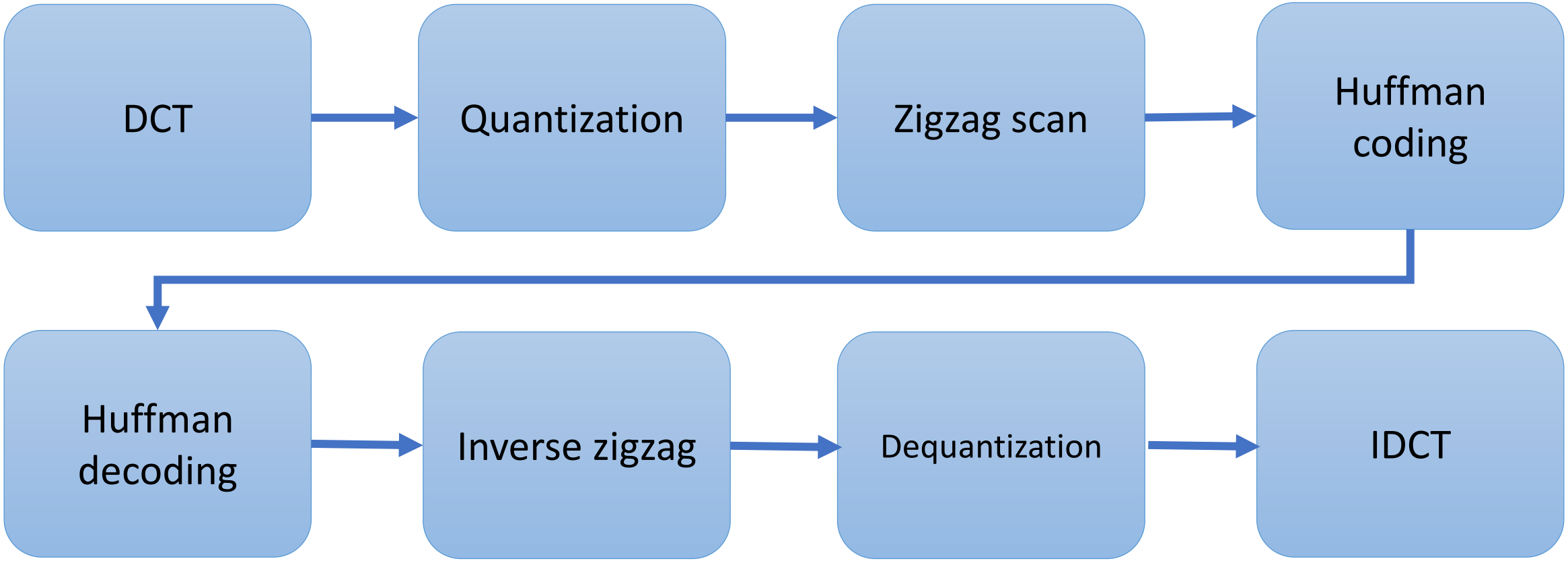
Video Compression

Spatial Coding

Objectives


DCT
Quantization
Zigzag scan
Huffman coding
Huffman decoding
Inverse zigzag
Dequantization
IDCT

Provided Files

Implementation:

- encoder.py (template)
- decoder.py(template)

Provided Codec Files

- codec_parameters.py
- evaluate.py

Input Images

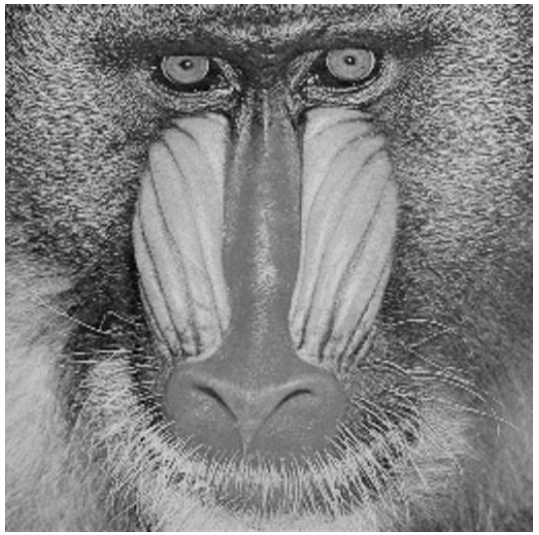
- lena_gray.png
- mandril_gray.png

Reference Outputs

- lena_reference_output.txt
- mandril_reference_output.txt

Input and Block Order

- All of the input images(include hidden case) have the following properties:
  - Size:512×512
  - 8-bit grayscale images.
  - Pixel values range from 0 to 255.
- Split each image into non-overlapping 8×8 blocks.
- Process blocks in raster order: left to right, then top to bottom.
- Do not resize, crop, denoise, or otherwise preprocess the input image.





Running the Encoder and Decoder

Encoder

python encoder.py <input_png> <output_bin> [--trace <trace_file>] [--block-index <index>]

e.g. python encoder.py lena_gray.png output.bin --trace encoder_trace.txt --block-index 0

Decoder

python decoder.py <input_bin> <output_png> [--trace <trace_file>] [--block-index <index>]

e.g. python decoder.py output.bin reconstructed.png --trace decoder_trace.txt --block-index 0

Evaluation

python evaluate.py <original_png> <reconstructed_png> <bitstream_bin>

e.g. python evaluate.py lena_gray.png reconstructed.png output.bin

DCT& IDCT

Level shift

Before performing the DCT, subtract 128 from all pixels

After IDCT, add 128 back to every reconstructed pixel value. Round to the nearest integer and clip the result to the range 0–255.

Orthonormal DCT-II

The provided file **codec_parameters.py** contains the 8 × 8 orthonormal DCT basis matrix C.

The matrix is constructed using:

$$\alpha(0) = \frac{1}{\sqrt{8}}, \qquad \alpha(k) = \sqrt{\frac{2}{8}}, \qquad k > 0$$

$$C(u,x) = \alpha(u)\cos\left(\frac{(2x+1)u\pi}{16}\right)$$

$$\text{DCT: } F = CfC^T, \qquad \text{IDCT: } \hat{f} = C^T\hat{F}C$$

DCT& IDCT

Separable 2D DCT and IDCT

Apply DCT in two 1D stages:

1. Apply 1D DCT to every row:

$$R = f\,C^{\mathrm{T}}$$

2. Apply 1D DCT to every column:

$$F = C\,R$$

Apply IDCT in the reverse order:

1. Apply 1D IDCT to every column:

$$M = C^{\mathrm{T}}\hat{F}$$

2. Apply 1D IDCT to every row:

$$\hat{f} = M\,C$$

Implementation Requirements

1. Use the provided DCT basis matrix C.
2. Implement DCT and IDCT using explicit arithmetic or matrix multiplication.
3. NumPy may be used for array operations and matrix multiplication.
4. Do not use built-in DCT or IDCT functions, such as scipy.fft.dct, scipy.fft.idct, cv2.dct, or cv2.idct.

Quantization

Quantization:

$$QF(u,v) = rint\left(\frac{F(u,v)}{Q(u,v)}\right)$$

Divide each DCT coefficient (F) by the corresponding number in the matrix (Q), and then use rint (Round to Integer) to round to the nearest integer.

Dequantization

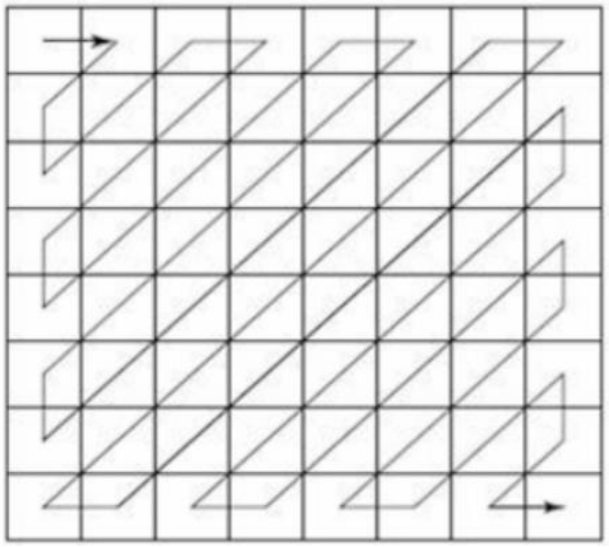
$$\hat{F}(u,v) = QF(u,v)Q(u,v).$$

Multiply the quantized number back by the original divisor.

Quantization matrix

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Zigzag scan



0, 1, 8, 16, 9, 2, 3, 10,
17, 24, 32, 25, 18, 11, 4, 5,
12, 19, 26, 33, 40, 48, 41, 34,
27, 20, 13, 6, 7, 14, 21, 28,
35, 42, 49, 56, 57, 50, 43, 36,
29, 22, 15, 23, 30, 37, 44, 51,
58, 59, 52, 45, 38, 31, 39, 46,
53, 60, 61, 54, 47, 55, 62, 63

Encoder: Convert the quantized 8x8 matrix into a 1D sequence of 64 coefficients in this order.

Decoder: Perform an inverse zigzag to place the 64 coefficients back into their correct 2D positions.

Huffman Coding& Token Payload Format

For a coefficient value v, define its magnitude category as:

$$s = \begin{cases} 0, v = 0 \\ \lfloor \log_2 |v| \rfloor + 1, v \neq 0 \end{cases}$$

EOB (End of Block): All remaining coefficients in the current block are zero.

Category (s)	Prefix code (length)	Amplitude bits	Value range
0	00 (2)	0	0
1	010 (3)	1	−1, 1
2	011 (3)	2	−3 to −2, 2 to 3
3	100 (3)	3	−7 to −4, 4 to 7
4	101 (3)	4	−15 to −8, 8 to 15
5	110 (3)	5	−31 to −16, 16 to 31
6	1110 (4)	6	−63 to −32, 32 to 63
7	11110 (5)	7	−127 to −64, 64 to 127
8	111110 (6)	8	−255 to −128, 128 to 255
9	1111110 (7)	9	−511 to −256, 256 to 511
10	11111110 (8)	10	−1023 to −512, 512 to 1023
11	111111110 (9)	11	−2047 to −1024, 1024 to 2047
EOB	111111111 (9)	—	—

Huffman Coding& Token Payload Format

Amplitude bits for VALUE

- For s = 0, VALUE(0) has no amplitude bits.
- For positive v, write v as an unsigned s-bit binary integer.
- For negative v, write v + (2^s - 1) as an unsigned s-bit binary integer.
- All bits are written MSB-first.

Examples:

VALUE(0) = 00
VALUE(1) = 010 | 1
VALUE(-1) = 010 | 0
VALUE(5) = 100 | 101
VALUE(-2) = 011 | 01
EOB = 111111111

Huffman Decoding

1. Initialize
Create a **64-entry zigzag array**
Initialize every entry to **0**

2. Decode symbols
Decode the prefix as either
VALUE(category s) or **EOB**
Continue until **64 values** are produced, or an **EOB** is read

3. Decode a VALUE
Read **s amplitude bits**
s = 0 → value = 0
Otherwise:

$$v = \begin{cases} a, a \geq 2^{\{s-1\}} \\ a - 2^s + 1, a < 2^{\{s-1\}} \end{cases}$$

Store v at the current zigzag position, then advance by one

4. Handle EOB
Stop decoding the current block
All remaining zigzag entries **remain zero**

Bitstream Format

4-byte header | Huffman payload | 0～7 final padding bits

Part	Location	Length	Description
Header	Bytes 0–3	32 bits	Stores valid_payload_bits: the exact number of valid bits in the payload only, excluding the header and final padding bits. Stored as an unsigned **big-endian** integer.
Payload	After header	valid_payload_bits	Concatenated VALUE / EOB codes for all blocks in raster order; write bits **MSB-first**.
Final padding	End of file	0–7 bits	Zero bits used only for byte alignment; not included in the payload and not decoded.

$$N_{padding} = \left(-N_{payload}\right) \bmod 8.$$

Metrics

1. Mean Squared Error (MSE)

Applies to the original image (I) and reconstructed image ($\hat{I}$) with size $H \times W$.

$$MSE = \frac{1}{HW} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \left(I(x,y) - \hat{I}(x,y) \right)^2$$

2. Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 10 \log_{10} \left(\frac{255^2}{MSE} \right) dB \, .$$

3. Bits Per Pixel (BPP)

$$bpp = \frac{\text{actual .bin file size (bytes)} \times 8}{H \times W}$$

Must be calculated using the **actual .bin file size**. Do not calculate BPP based solely on payload bits or theoretical average code lengths.

TODO

encoder.py

- transform_block()
  - Level shift, DCT, quantization, coefficient-range validation, and zigzag scan
- encode_block_symbols(zigzag)
  - Internal VALUE(0) symbols and correct EOB handling
- write_symbol(...)
  - VALUE/EOB bitstream writing

decoder.py

- read_value(reader, s)
  - Signed coefficient recovery from amplitude bits
- read_block(reader)
  - VALUE/EOB decoding into 64 zigzag coefficients
- reconstruct_block(zigzagged)
  - Inverse zigzag, dequantization, IDCT, level restoration, rounding, and clipping

Please do not modify the output txt format.

Grading Policy

70%: Standard Implementation

Two test cases are provided, and hidden test cases will be used.

Phase	Reference Output to Match
DCT(14%)	AFTER ROW-WISE 1D DCT, DCT COEFFICIENTS
Quantization(14%)	QUANTIZED COEFFICIENTS
Zigzag(14%)	ZIGZAG COEFFICIENTS
Huffman(14%)	VALUE / EOB SYMBOLS, BIT COUNTS
Final Result(14%)	Result from evaluate.py

Use **lena_reference_output.txt** and **mandril_reference_output.txt** to compare the intermediate results for block index 0 and the final evaluation metrics.

Your implementation does not need to be bit-for-bit identical to the reference.

Allowed differences:**Floating-point values:** 0.01, **PSNR:** 0.01 dB, **MSE:** 0.01, **BPP:** 0.0001, **Payload length:** $\max(16, \lceil 0.0001\ W \times H\ \rceil)$ bits

Grading Policy

30%: Lab Report

Please conduct the following ablation experiments ,observe whether the specified metrics change, and analyze the reason:

- (10%)Custom modifications to the Quantization table -> bpp, PSNR
- (10%)Zigzag vs. Raster-> bpp, PSNR
- (10%)Two stage 1D DCT vs. Direct 2D DCT -> Time, bpp

(It is recommended to measure DCT computation time only. Use any timing tool, such as time.perf_counter().)

$$Direct\ 2D\ DCT\colon\ F(u,v) = \sum_{x=0}^{7}\sum_{y=0}^{7} C(u,x)f(x,y)C(v,y)$$

Implement the direct 2D DCT using the double-summation formula, not C @ f @ C.T

For ablation experiments, you may modify any provided source file or create additional files as needed. Include the experimental results and analysis in your report only. Do not submit ablation code.

Submission

Deadline: 2026/10/19(Monday) 23:59 **(Late submissions will not be accepted)**

Submit one ZIP file containing your encoder, decoder, and report.

HW1_[STUDENT ID].zip

```
├── encoder.py
├── decoder.py
└── HW1_[STUDENT ID].pdf
```

Notice:

- Do **not include an additional folder** inside the ZIP file**.**
- Do **not change the provided output format**, including trace section titles, field names, and ordering.
- Submit only the standard implementation. Do not submit ablation variants.
- Format violations may result in deductions 10 points

How to Create the Submission ZIP

- Place encoder.py, decoder.py, and HW1_[STUDENT ID].pdf in the same folder.
- **Select the three files**, then compress them into HW1_[STUDENT ID].zip.
- Open the ZIP to confirm that the three files appear directly inside it.



You may also use scripts, command-line tools, or other methods to create the ZIP file. Ensure that the required files appear directly at the root of the ZIP file, without an additional enclosing folder.